

前后端目录规范

本文档用于统一本项目的目录组织方式，目标是让组员一看就知道东西该放哪里，避免后期出现“代码到处都是、没人知道谁负责”的情况。

适用范围：校园二手交易平台微信小程序前端、FastAPI 后端、数据库建模、测试与联调相关代码。

当前仓库的实际分层是：微信小程序前端统一放在 [miniprogram/](#)，后端统一放在 [server/](#)，文档与规范放在 [docs/](#)。微信开发者工具建议直接打开 [miniprogram/](#) 作为前端工程根目录。

1. 总原则

1. 按业务域分，不按个人习惯分。
2. 路由层只负责接收请求、调用业务、返回结果，不写复杂业务逻辑。
3. 通用能力集中管理，禁止散落在各模块里重复实现。
4. 状态、权限、错误码、接口字段等关键定义必须统一入口。
5. 一个目录只承担一种职责，避免“一个文件既管接口又管逻辑又管数据库”。
6. 新功能先判断属于接口层、业务层、数据层还是通用层，再决定目录位置。

2. 后端目录规范

后端以 [server/app/](#) 为核心。建议目录结构如下：

```
server/app/  
  main.py  
  api/  
    routers/  
    deps/  
    __init__.py  
  core/  
    response.py  
    config.py  
    status.py  
    exceptions.py  
    security.py  
    constants.py  
    __init__.py  
  schemas/  
    common.py  
    auth.py  
    users.py  
    products.py  
    orders.py  
    reports.py  
    admin.py  
    __init__.py  
  models/  
    user.py  
    product.py
```

```
order.py
report.py
admin_log.py
__init__.py
crud/
  user.py
  product.py
  order.py
  report.py
  admin.py
  __init__.py
services/
  auth_service.py
  user_service.py
  product_service.py
  order_service.py
  report_service.py
  admin_service.py
  __init__.py
db/
  session.py
  base.py
  init_db.py
  seed.py
  __init__.py
utils/
  pagination.py
  validators.py
  __init__.py
tests/
  test_auth.py
  test_users.py
  test_products.py
  test_orders.py
  test_reports.py
  test_admin.py
```

2.1 main.py

- 只做应用启动、全局中间件、路由挂载。
- 不写业务逻辑。
- 不直接处理复杂异常转换。

2.2 api/routers/

放各模块路由入口，例如认证、用户、商品、订单、举报、后台管理。

职责：

- 接收请求参数。
- 调用 service 层。
- 返回统一响应。

禁止内容：

- SQL 语句。
- 大段业务判断。
- 状态机规则硬编码散写。

2.3 `api/deps/`

放依赖注入相关内容，例如：

- 当前登录用户。
- 当前管理员。
- 数据库会话。
- 分页参数。
- 权限校验依赖。

2.4 `core/`

放项目级基础能力，例如：

- 统一响应。
- 配置读取。
- 状态枚举。
- 异常定义。
- 安全和鉴权。
- 常量定义。

其中状态枚举建议集中放在 `server/app/core/status.py` 或同类文件里统一维护。

2.5 `schemas/`

放请求体、响应体、数据传输模型。

建议按业务模块拆分文件，不要把所有模型长期堆在 `common.py`。

2.6 `models/`

放数据库 ORM 模型。

原则：

- 一张表一个模型，或一个业务域一组模型。
- 模型只描述表结构，不写复杂业务逻辑。

2.7 `crud/`

放数据库访问层。

职责：

- 查询。
- 新增。

- 更新。
- 删除。
- 条件过滤。

不负责业务编排。

2.8 `services/`

放业务逻辑层。

例如：

- 登录后创建用户。
- 发布商品后生成审核记录。
- 下单时检查状态是否允许。
- 举报处理后同步更新相关表。

2.9 `db/`

放数据库连接、Session、初始化与种子数据。

后续如果引入迁移工具，也优先放在这个目录附近统一管理。

2.10 `utils/`

放后端通用工具函数，例如：

- 分页工具。
- 校验工具。
- 格式转换工具。
- 与具体业务无关的小工具。

2.11 `tests/`

放后端测试代码。

建议按业务域拆分测试文件，方便定位问题。

3. 后端硬性约定

1. 路由文件里不允许写核心业务逻辑。
2. 状态枚举、错误码和权限定义必须集中管理。
3. 数据库访问必须经过 `service` 或 `crud`，不要路由直接碰数据库。
4. 管理员相关能力必须独立隔离，不能和普通用户逻辑混写。
5. 所有新增字段先看 `schema`，再看 `model`，再看接口。
6. 业务逻辑优先收敛到 `services/`，不要散在 `router` 里。

4. 前端目录规范

前端以 `miniprogram/` 作为微信小程序工程根目录。建议目录结构如下：

```
miniprogram/  
  app.js  
  app.json  
  app.wxss  
  project.config.json  
  pages/  
    home/  
    list/  
    detail/  
    post/  
    user/  
    admin/  
  components/  
    item-card/  
    search-bar/  
    empty-state/  
    loading-state/  
    error-tip/  
    confirm-dialog/  
  services/  
    auth.js  
    user.js  
    product.js  
    order.js  
    report.js  
    admin.js  
  utils/  
    api.js  
    storage.js  
    validator.js  
    formatter.js  
    constants.js  
  assets/  
    images/  
    icons/  
  styles/  
    common.wxss  
    variables.wxss
```

4.1 pages/

放页面入口和页面级状态管理。

原则：

- 页面可以写少量交互逻辑。
- 页面不直接散写复杂请求封装。
- 页面不直接承载所有公共工具逻辑。

4.2 components/

放可复用组件。

例如：

- 商品卡片。
- 搜索框。
- 空状态。
- 加载态。
- 错误提示。
- 确认弹窗。

组件只做展示和轻交互，不写完整业务流程。

4.3 services/

放页面调用的接口封装。

建议按业务域拆分：

- `auth.js` 负责登录相关接口。
- `user.js` 负责用户相关接口。
- `product.js` 负责商品相关接口。
- `order.js` 负责订单相关接口。
- `report.js` 负责举报相关接口。
- `admin.js` 负责后台相关接口。

页面优先调用 `services/`，不要直接到处写 `wx.request`。

4.4 utils/

放通用工具和全局常量。

建议职责如下：

- `api.js`：统一请求入口。
- `storage.js`：统一本地存储。
- `validator.js`：表单校验。
- `formatter.js`：时间、价格、状态文案格式化。
- `constants.js`：状态映射、页面枚举、公共常量。

4.5 assets/

放图片、图标、默认占位图等静态资源。

4.6 styles/

放全局样式和变量。

不要每个页面都自己定义一套颜色、字体和空状态样式。

5. 前端硬性约定

1. 页面不直接硬编码后端接口地址。
2. 页面不直接拼复杂请求参数，统一走 `services/`。
3. 状态文案、颜色和空状态统一管理，避免每页一套。
4. 组件不写业务流程，只负责展示和轻交互。
5. 登录态统一通过 `utils/storage.js` 管理。
6. 前端展示状态必须和后端状态枚举保持一致。

6. 跨端统一约定

6.1 状态定义统一入口

状态枚举建议统一放在后端的 `server/app/core/status.py` 中集中维护，后端业务代码、数据库迁移脚本和 OpenAPI 草案都以这份定义为准。前端如果需要展示中文文案或颜色，只做显示层映射，不要自己重新发明一套状态值。

6.2 接口请求统一入口

前端所有请求先进入 `utils/api.js`，再由 `services/` 按模块调用。

6.3 业务逻辑统一入口

后端所有核心业务先走 `services/`，再落到 `crud/`，最后到 `models/`。

6.4 命名统一

- 状态命名统一。
- 时间字段统一。
- 接口字段统一。
- 数据库字段统一。

4.7 `project.config.json`

这是微信开发者工具识别工程的重要配置文件。建议放在 `miniprogram/` 根目录，方便直接用微信开发者工具打开该目录进行开发。

7. 推荐的落地顺序

1. 先定后端目录层次。
2. 再把前端 `services` 和 `utils` 分出来。
3. 再补状态定义和常量映射。
4. 最后开始往各目录里填业务代码。

8. 备注

如果后续发现某个目录职责不清，优先修改这份文档，再同步调整项目结构，避免不同文档和代码之间打架。